Fysik PG 10060

Forårs-forelæsning #7

Bevægelsesligninger

Forelæser:
Alexander Bagger,
Lektor, DTU Fysik

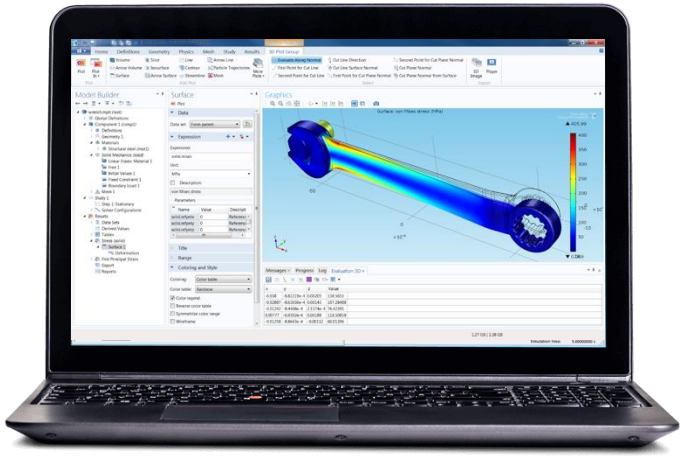
Kursusoverblik

Overblik over forårets forelæsninger, **foreløbig** plan:

Uge 1: Kinematik 1D (Kap. 3)
Uge 2: Kinematik 2D (Kap. 4)
Uge 3: Usikkerhed
Uge 4: Eksperimenter
Uge 5: Kræfter 1 (Kap. 5)
Uge 6: Kræfter 2 (Kap. 6)
Uge 7: Bevægelsesligninger (Kap. 15)
Uge 8: Arbejde Og Energi (Kap. 7)
Påske (30/3)
Påske (6/4)
Uge 9: Energibevarelse (Kap. 8)
Uge 10: Stød Og Impulsbevarelse (Kap. 9)
Uge 11: Rotation 1 (Kap. 10)
Uge 12: Rotation 2 (Kap. 11)
Uge 13: Energitema 1

Dagens forelæsning:

Uge 7: Bevægelsesligninger



- Hvorfor simuleringer?
- Eulers metode
- Numeriske løsning (scipy's solve_ivp)
- Events i simuleringer
  - Svingninger (fjeder) eksempel
  - Luftmodstand eksempel

Repetition (Kinematik 1D): Relevante formler

Konstant acceleration

Equation	Contains
$v = v_0 + at$	v, a, t; no x
$x = x_0 + \frac{1}{2}(v_0 + v)t$	x, v, t; no a
$x = x_0 + v_0 t + \frac{1}{2}at^2$	x, a, t; no v
$v^2 = v_0^2 + 2a(x - x_0)$	x, v, a; no t

General acceleration

$$\frac{dv}{dt} = a \Rightarrow \qquad v(t) = \int_0^t a(t)dt + v_0$$

$$\frac{dx}{dt} = v \Rightarrow \qquad x(t) = \int_0^t v(t)dt + x_0$$

Frit fald acceleration

$$v = v_0 - gt \ (positive\ upward)$$

$$y = y_0 + v_0 t - \frac{1}{2}gt^2 \ (positive\ upward)$$

Repetition (Kinematik 2D): Relevante formler

- **Projektilbevægelse (skrå kast)**

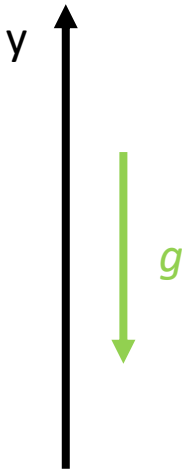
Affyring/ Landing Horizontal

Tid $T_{tof} = \frac{2v_0 \sin(\theta_0)}{g}$

Bane ligning: $y = \tan(\theta_0)\, x - \left[\frac{g}{2v_0^2 \cos^2(\theta)}\right] x^2$

Højeste punkt: $y = \frac{(v_0 \sin(\theta))^2}{2g}$

Længde: $\mathrm{R} = \frac{v_0^2 \sin(2\theta_0)}{g}$



Generelt

$\mathrm{x}(t) = v_{0x} t$

$y(t) = v_{0y} t - \frac{1}{2} g t^2$

$\mathrm{v}_x(t) = v_{0x}$

$\mathrm{v}_y(t) = v_{0y} - gt$

$\mathrm{v}_y^2 = v_{0y}^2 - 2g(y - y_0)$

$v^2 = v_0^2 - 2g(y - y_0)$

$\mathrm{y(x)} = y_0 + \tan(\theta)\, x - \frac{g}{2v_0^2 cos^2(\theta)} x^2$

- **Cirkel bevægelse**

$$v = \frac{2\pi r}{T}$$

$$a_c = \frac{v^2}{r}$$

$$a_T = \frac{d|\boldsymbol{v}|}{dt}$$

- **Relativ bevægelse**

$$\boldsymbol{v_{PS} = v_{PS'} + v_{S'S}}$$

$$\boldsymbol{a_{PS} = a_{PS'} + a_{S'S}}$$

- **Trigonometriske formler**

Repetition (Usikkerhed): Relevante formler

Angivelse af usikkerhed

x	bedste bud på størrelse
δx	usikkerhed, positiv størrelse
$x \pm \delta x$	absolut usikkerhed
$\frac{\delta x}{\lvert x \rvert}$	relativ usikkerhed

Måleserie

Standardafgivelsen er $\delta x = \sqrt{\frac{\sum(\bar{x}-x_i)^2}{N-1}}$ (målingerne er en stikprøve)

Usikkerheden på gennemsnittet, $\bar{x}$, er $\delta\bar{x} = \frac{\delta x}{\sqrt{N}}$

-> standard deviation of the mean (SDOM).

Normal fordeling

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}$$

$P(\mu - \delta x \leq x \leq \mu + \delta x) = 0.68$
$P(\mu - 2\delta x \leq x \leq \mu + 2\delta x) = 0.95$
$P(\mu - 3\delta x \leq x \leq \mu + 3\delta x) = 0.99$

Tilfældige og systematiske usikkerheder

$$\delta x = \sqrt{\left(\delta x_{\text{tilfældige}}\right)^2 + \left(\delta x_{\text{systematiske}}\right)^2}$$

Beregninger med størrelser med usikkerhed

Beregn $z = x \pm y$

Uafhængige usikkerheder: $z \pm \delta z = x \pm y \pm \sqrt{\delta x^2 + \delta y^2}$

Afhængige usikkerheder: $z \pm \delta z = x \pm y \pm \delta x + \delta y$

Beregn $z = x \cdot y$

Uafhængige usikkerheder (relativ) $\frac{\delta z}{|z|} = \sqrt{\left(\frac{\delta x}{|x|}\right)^2 + \left(\frac{\delta x}{|x|}\right)^2}$

Afhængige usikkerheder (relativ) $\frac{\delta z}{|z|} = \frac{\delta x}{|x|} + \frac{\delta y}{|y|}$

Fejlophobningsloven

$z = f(x, y), x \pm \delta x,\ y \pm \delta y$

Uafhængige usikkerheder: $\delta z = \sqrt{\left(\frac{\partial f}{\partial x}\delta x\right)^2 + \left(\frac{\partial f}{\partial y}\delta y\right)^2}$

Afhængige usikkerheder: $\delta z = \left|\frac{\partial f}{\partial x}\right| \delta x + \left|\frac{\partial f}{\partial y}\right| \delta y$

Repetition (Eksperimenter): Relevante formler

$$\text{std}_{\text{score}} = \frac{\text{x} - \bar{x}}{\delta \text{x}}$$

$\text{std}_{\text{score}} < 2$ $\quad Good$

$2 < \text{std}_{\text{score}} < 3$ $\quad Grey\ zone$

$3 < \text{std}_{\text{score}}$ $\quad reject$

Sammenligning af to målinger

$$\text{std}_{\text{score}} = \frac{|z|}{\delta z}$$

$$z = x - y$$

$$\delta z = \sqrt{\delta x^2 + \delta y^2}$$

Naturlige skalaer

$\mu = m$ (masse)
$\lambda = l$ (længde)
$\tau = \sqrt{l/g}$ (tidsdimensionen)
$\alpha = \lambda/\tau^2$ = l/(l/g) (acceleration)

$t = f(m, g, h, k)$ Model

	m	g	h	k
M	1	0	0	1
L	0	1	1	0
T	0	−2	0	−1

Dimensionsmatricen

$v_2 = f(v_1, a, x)$ Model

$\frac{v_2}{v_1} = F\left(\frac{ax}{v_1^2}\right)$ Dimensionsløs formulering af modellen

Repetition (Newtons love): Relevante formler

Newtons love:

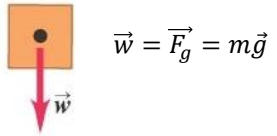
N1:
$$\sum F = 0 \implies v = konstant$$

N2:
$$\sum F = ma$$

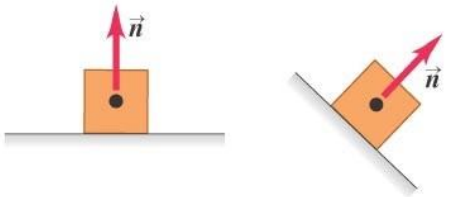
N3:
$$\overrightarrow{F_{AB}} = -\overrightarrow{F_{BA}}$$

Fem vigtige kræfter i mekanik

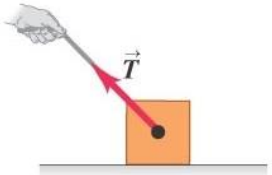
Tyngdekraft



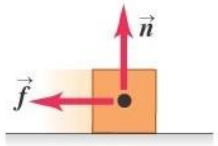
Normalkraft



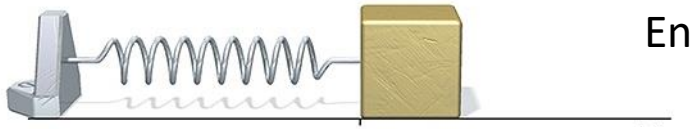
Snorkraft



Gnidningskraft



Fjederkraft



Enheden for kraft: N = kg·m/s^2

Repetition (kræfter 2): Relevante formler

- **Hooke's lov for fjederkraften:**

$$F_{\text{fjeder}} = -k \cdot (x - x_0)$$

$f_k = \mu_k \, n$ Kinematisk gnidning

$f_s \leq \mu_s \, n$ Statisk gnidning

$F_d = b \cdot v^n$ Luft- og vandmodstand (generelt)

$f = D \cdot v^2$ Luft- og vandmodstand (drag)

$$D = \frac{1}{2} \cdot \rho_{\text{luft}} \cdot C_D \cdot A$$

- **Rundt object der falder I et medium (drag)**

$F_s = 6\pi r \eta v$ Stokes' law

- **Cirkel bevægelse**

$$\vec{r} = (x, y) = (r \cos\theta, r \sin\theta)$$

$$r = \sqrt{x^2 + y^2}$$

$$\theta = tan^{-1}\left(\frac{y}{x}\right)$$

$$\omega = \frac{d\theta}{dt}$$

$$\mathrm{v} = \mathrm{r}\,\omega$$

$$\frac{d\omega}{dt} = \alpha$$

$a_{\mathrm{t}} = \frac{dv}{dt} = r\,\alpha$ Bemærk: $a_{\mathrm{t}} = 0$ ved cirkelbevægelse med konstant fart!

$$a_{\mathrm{c}} = v\,\omega = r\omega^2 = \frac{v^2}{r}$$

$$v = \frac{omkreds}{periode} = \frac{2\pi R}{T}$$

Relevante formler

- **Eulers metode**

$$x(t+\Delta t) \approx x(t) + v(t)\Delta t$$

$$v(t+\Delta t) \approx v(t) + a(t)\Delta t$$

- **Eulers-Cromers metode**

$$v(t+\Delta t) \approx v(t) + a(t)\Delta t$$

$$x(t+\Delta t) \approx x(t) + v(t+\Delta t)\Delta t$$

- **Scipy solve_ivp**

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
t1 = 0.0
t2 = 10.0
m = 1.0
k = 10.0
def f(t,y):
    x,v = y
    return [v,-k*x/m]
sol = solve_ivp(f,[t1,t2],y0=[0.1,0.0],atol=1e-8,rtol=1e-8)
t = sol.t
x = sol.y[0]
v = sol.y[1]
```

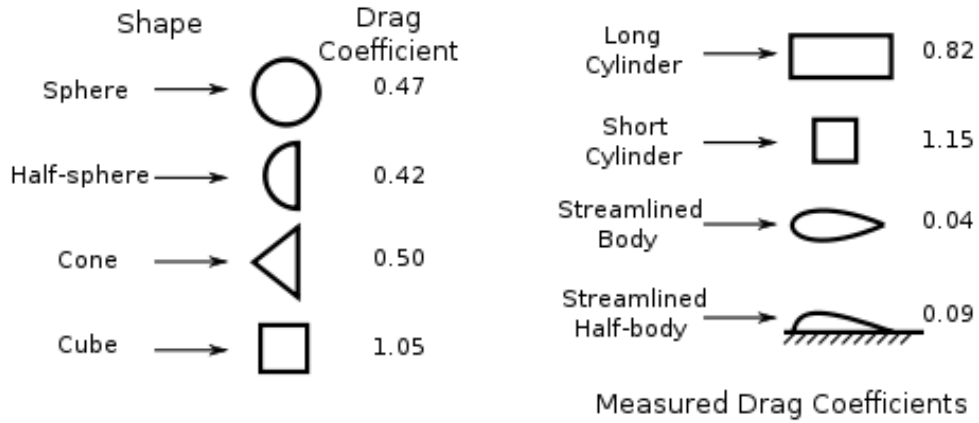
- **Events**

```python
def hit_ground(t,z):
    y,v = z
    return y
```

```python
hit_ground.terminal = True
hit_ground.direction = -1
```

Hvorfor simuleringer?

- ”Dovenskab”
- Analytisk løsning for kompliceret
- Ikke-lineære problemer
- ”Billig” optimering



Question slide

Which equation is this? (Select more)

$$m\ddot{x} + kx = 0$$

A. It is a differential equation

0%

B. A homogenous differential equation

0%

C. There exists one solution

0%

D. It is a first order differential equation

0%

E. It is a inhomogenous differential equation

0%

F. There is 1 first order term.

0%

G. It is a second order differential equation.

0%

Which equation is this? (Select more)

$$m\ddot{x} + kx = 0$$

A. It is a differential equation

0%

B. A homogenous differential equation

0%

C. There exists one solution

0%

D. It is a first order differential equation

0%

E. It is a inhomogenous differential equation

0%

F. There is 1 first order term.

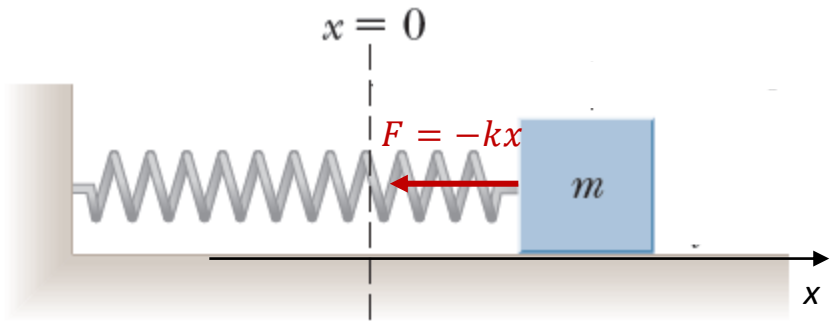
0%

G. It is a second order differential equation.

##.#0%

Ligninger opskrives som første ordens ligninger

$$m\ddot{x} + kx = 0$$



$$\dot{x} = v$$

$$\dot{v} = -\frac{k}{m}x$$

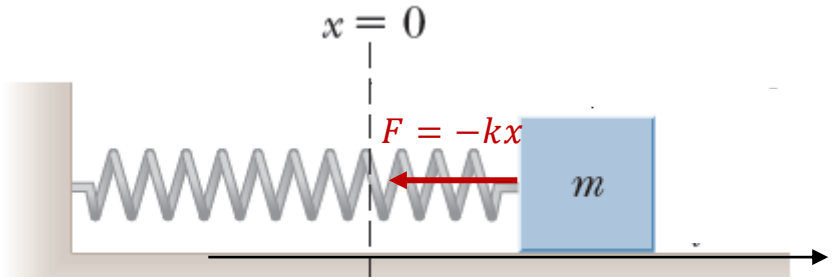
Eulers metode

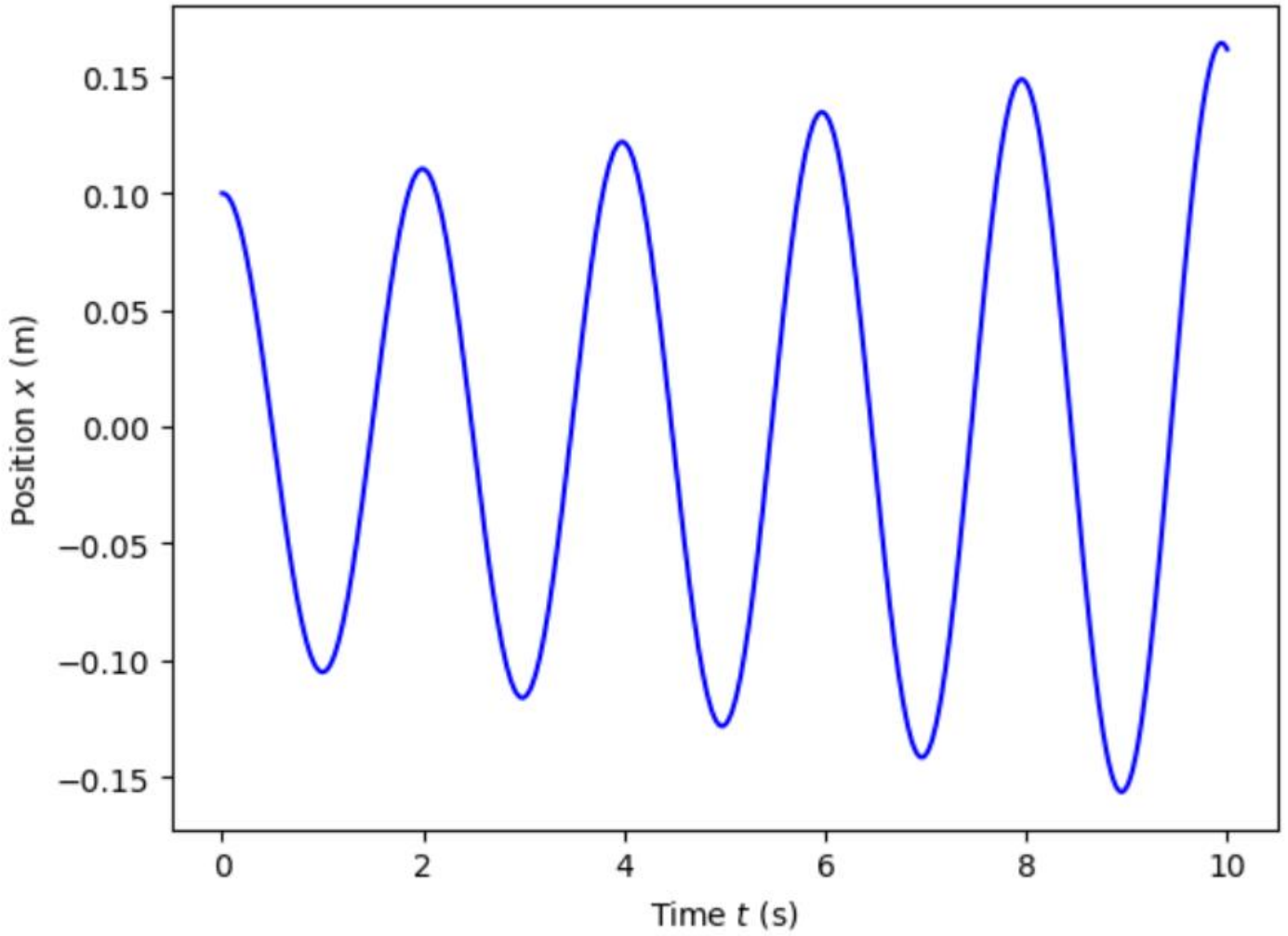
$$x(t + \Delta t) \approx x(t) + v(t)\Delta t$$

$$v(t + \Delta t) \approx v(t) + a(t)\Delta t$$

Eulers metode

```
t1 = 0.0
t2 = 10.0
N = 1000
t = np.linspace(t1,t2,N+1)
h = (t2-t1)/N
x = np.zeros(N+1)
v = np.zeros(N+1)
m = 1.0
k = 10.0
x[0] = 0.1
v[0] = 0.0
def a(x):
    return -k*x/m
for i in range(N):
    v[i+1] = v[i] + a(x[i]) * h
    x[i+1] = x[i] + v[i] * h
```





Eulers metode

$$x(t + \Delta t) \approx x(t) + v(t)\Delta t$$

$$v(t + \Delta t) \approx v(t) + a(t)\Delta t$$

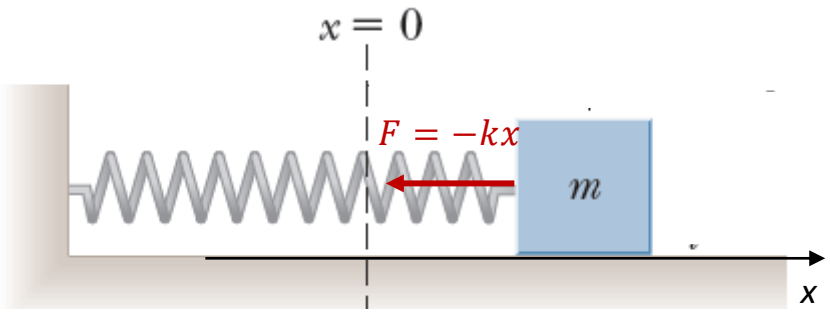
Modificeret Eulers metode

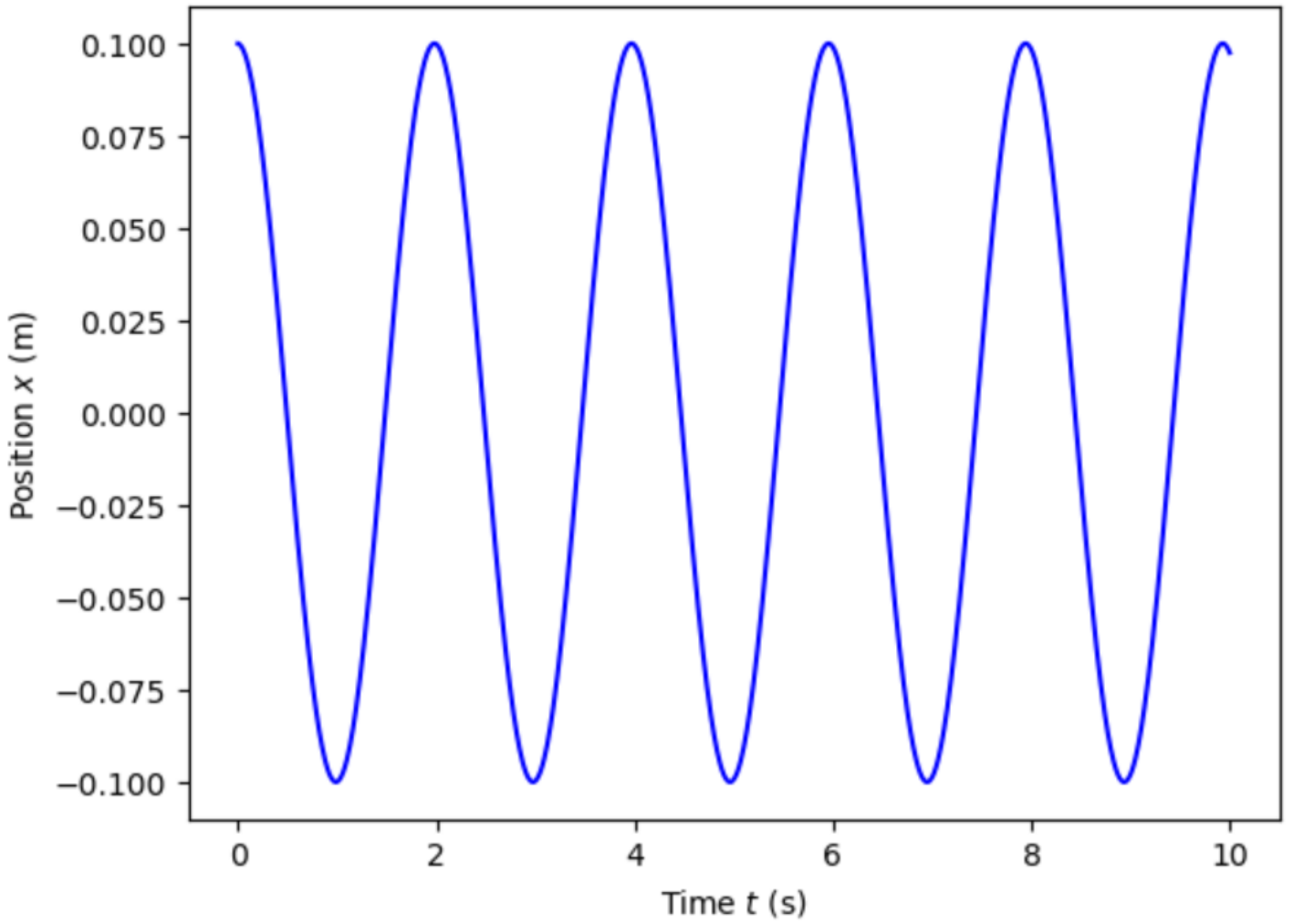
$$v(t + \Delta t) \approx v(t) + a(t)\Delta t$$

$$x(t + \Delta t) \approx x(t) + v(t + \Delta t)\Delta t$$

Modificeret Eulers metode

```
t1 = 0.0
t2 = 10.0
N = 1000
t = np.linspace(t1,t2,N+1)
h = (t2-t1)/N
x = np.zeros(N+1)
v = np.zeros(N+1)
m = 1.0
k = 10.0
x[0] = 0.1
v[0] = 0.0
def a(x):
    return -k*x/m
for i in range(N):
    v[i+1] = v[i] + a(x[i]) * h
    x[i+1] = x[i] + v[i+1] * h
```





solve_ivp metode

```python
from scipy.integrate import solve_ivp
t1 = 0.0
t2 = 10.0
m = 1.0
k = 10.0
def f(t,y):
    x,v = y
    return [v,-k*x/m]
sol = solve_ivp(f,[t1,t2],y0=[0.1,0.0])
display(sol)
t = sol.t
x = sol.y[0]
v = sol.y[1]
```

docs.scipy.org

SciPy v1.12.0 Manual

Installing | User Guide | API reference | Building from source | Development | Release notes

Search the docs ...

- scipy
- scipy.cluster
- scipy.constants
- scipy.datasets
- scipy.fft
- scipy.fftpack
- scipy.integrate
- scipy.interpolate
- scipy.io
- scipy.linalg
- scipy.misc
- scipy.ndimage
- scipy.odr
- scipy.optimize
- scipy.signal
- scipy.sparse
- scipy.spatial
- scipy.special
- scipy.stats

scipy.integrate.solve_ivp

scipy.integrate.solve_ivp(*fun*, *t_span*, *y0*, *method='RK45'*, *t_eval=None*, *dense_output=False*, *events=None*, *vectorized=False*, *args=None*, ***options*)

Solve an initial value problem for a system of ODEs. [source]

This function numerically integrates a system of ordinary differential equations given an initial value:

```
dy / dt = f(t, y)
y(t0) = y0
```

Here t is a 1-D independent variable (time), y(t) is an N-D vector-valued function (state), and an N-D vector-valued function f(t, y) determines the differential equations. The goal is to find y(t) approximately satisfying the differential equations, given an initial value y(t0)=y0.

Some of the solvers support integration in the complex domain, but note that for stiff ODE solvers, the right-hand side must be complex-differentiable (satisfy Cauchy-Riemann equations [11]). To solve a problem in the complex domain, pass y0 with a complex data type. Another option always available is to rewrite your problem for real and imaginary parts separately.

Parameters: **fun** : *callable*

Right-hand side of the system: the time derivative of the state `y` at time `t`. The calling signature is `fun(t, y)`, where `t` is a scalar and `y` is an ndarray with `len(y) = len(y0)`. Additional arguments need to be passed if `args` is used (see documentation of `args` argument). `fun` must return an array of the same shape as `y`. See *vectorized* for more information.

t_span : *2-member sequence*

Interval of integration (t0, tf). The solver starts with t=t0 and integrates until it reaches t=tf. Both t0 and tf must be floats or values interpretable by the float conversion function.

y0 : *array_like, shape (n,)*

Initial state. For problems in the complex domain, pass *y0* with a complex data type (even if the initial value is purely real).

What does solve_ivp return? (select more)

```
1  from scipy.integrate import solve_ivp
2  t1 = 0.0
3  t2 = 10.0
4  m = 1.0
5  k = 10.0
6  def f(t,y):
7      x,v = y
8      return [v,-k*x/m]
9  sol = solve_ivp(f,[t1,t2],y0=[0.1,0.0])
10 display(sol)
11 t = sol.t
12 x = sol.y[0]
13 v = sol.y[1]
```

A. t, y 0%

B. fft, ifft 0%

C. t_events, y_events 0%

D. Success, fail 0%

E. Seterror, errorstate 0%

F. Derivative, Model 0%

G. Sol, nfev 0%

What does solve_ivp return? (select more)

```
1  from scipy.integrate import solve_ivp
2  t1 = 0.0
3  t2 = 10.0
4  m = 1.0
5  k = 10.0
6  def f(t,y):
7      x,v = y
8      return [v,-k*x/m]
9  sol = solve_ivp(f,[t1,t2],y0=[0.1,0.0])
10 display(sol)
11 t = sol.t
12 x = sol.y[0]
13 v = sol.y[1]
```

A. t, y — 0%

B. fft, ifft — 0%

C. t_events, y_events — 0%

D. Success, fail — 0%

E. Seterror, errorstate — 0%

F. Derivative, Model — 0%

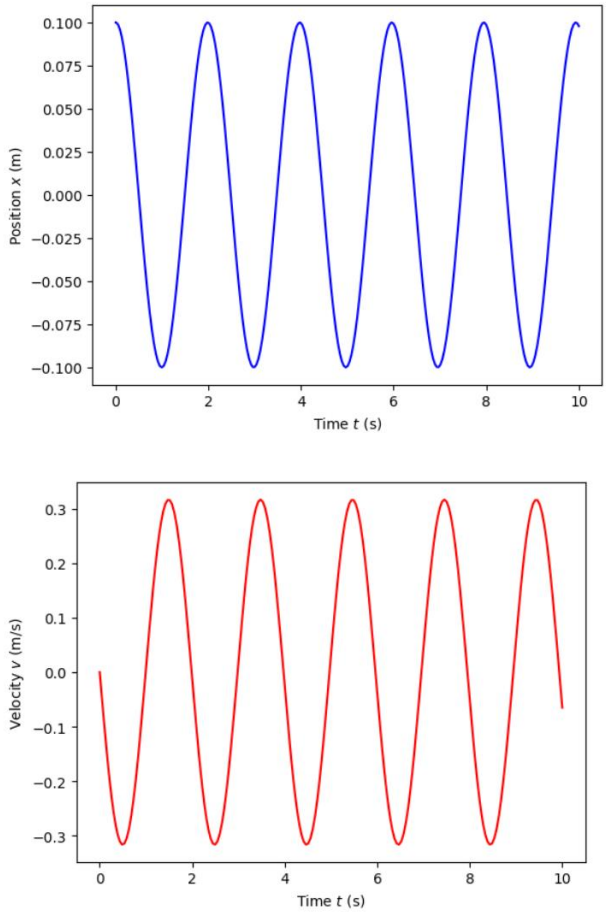
G. Sol, nfev — 0%

Forbedring af solve_ivp

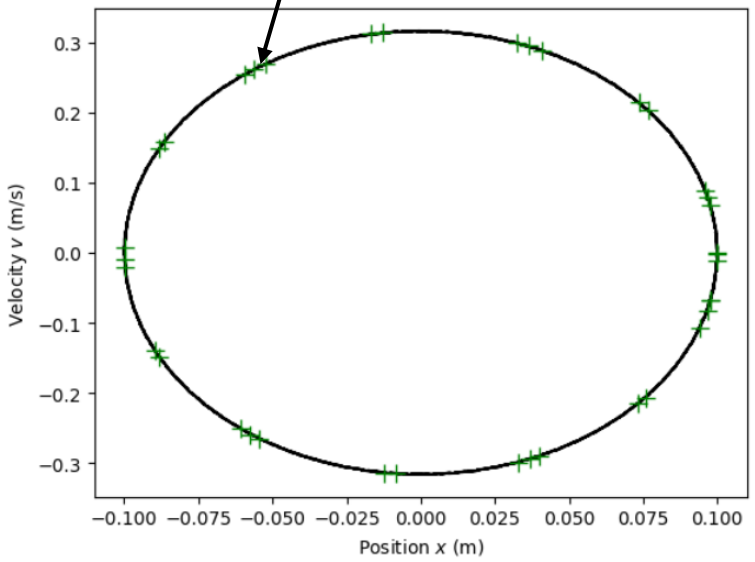
code input

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
t1 = 0.0
t2 = 10.0
m = 1.0
k = 10.0
def f(t,y):
    x,v = y
    return [v,-k*x/m]
sol = solve_ivp(f,[t1,t2],y0=[0.1,0.0],atol=1e-8,rtol=1e-8)
t = sol.t
x = sol.y[0]
v = sol.y[1]
```

data output

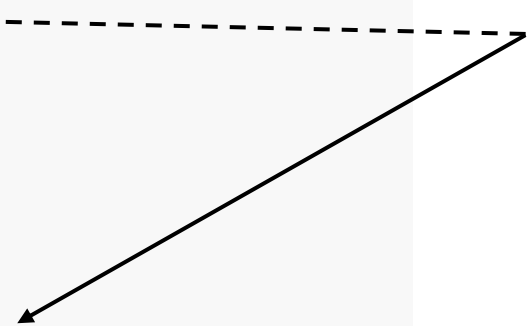


Green points are the previously computed.



Løsning med solve_ivp metode

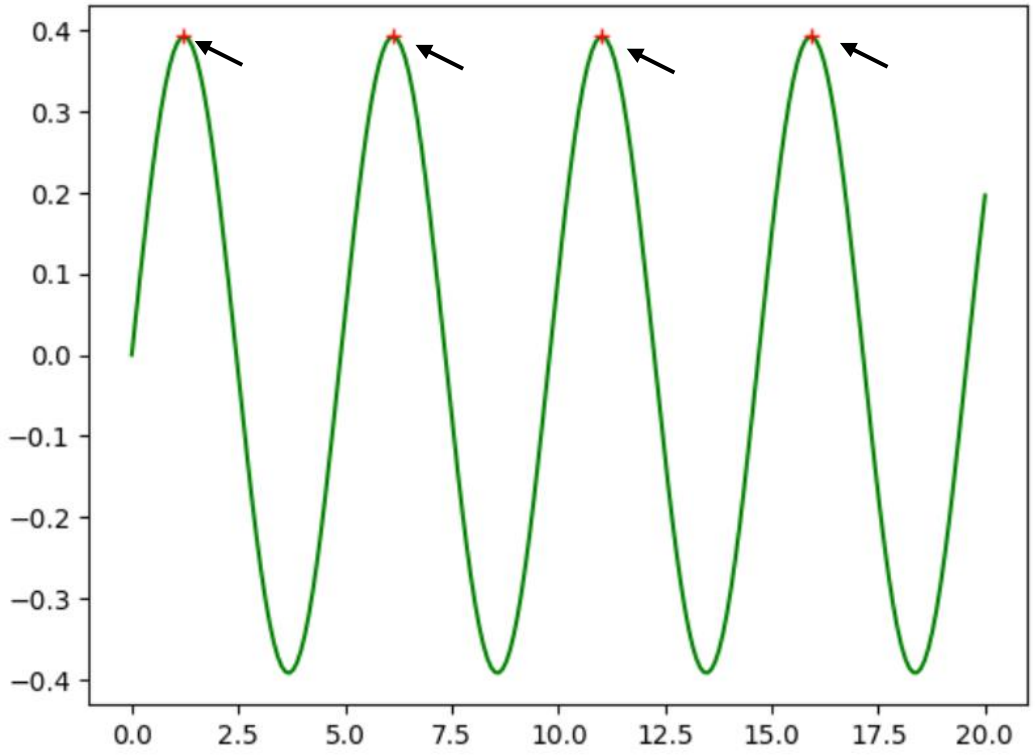
```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.integrate import solve_ivp
4  t1 = 0.0
5  t2 = 10.0
6  N = 1000
7  t = np.linspace(t1,t2,N+1)
8  m = 1.0
9  k = 10.0
10 def f(t,y):
11     x,v = y
12     return [v,-k*x/m]
13 sol = solve_ivp(f,[t1,t2],t_eval=t,y0=[0.1,0.0])
14 t = sol.t
15 x = sol.y[0]
16 v = sol.y[1]
```



Time points at which to compute solution.

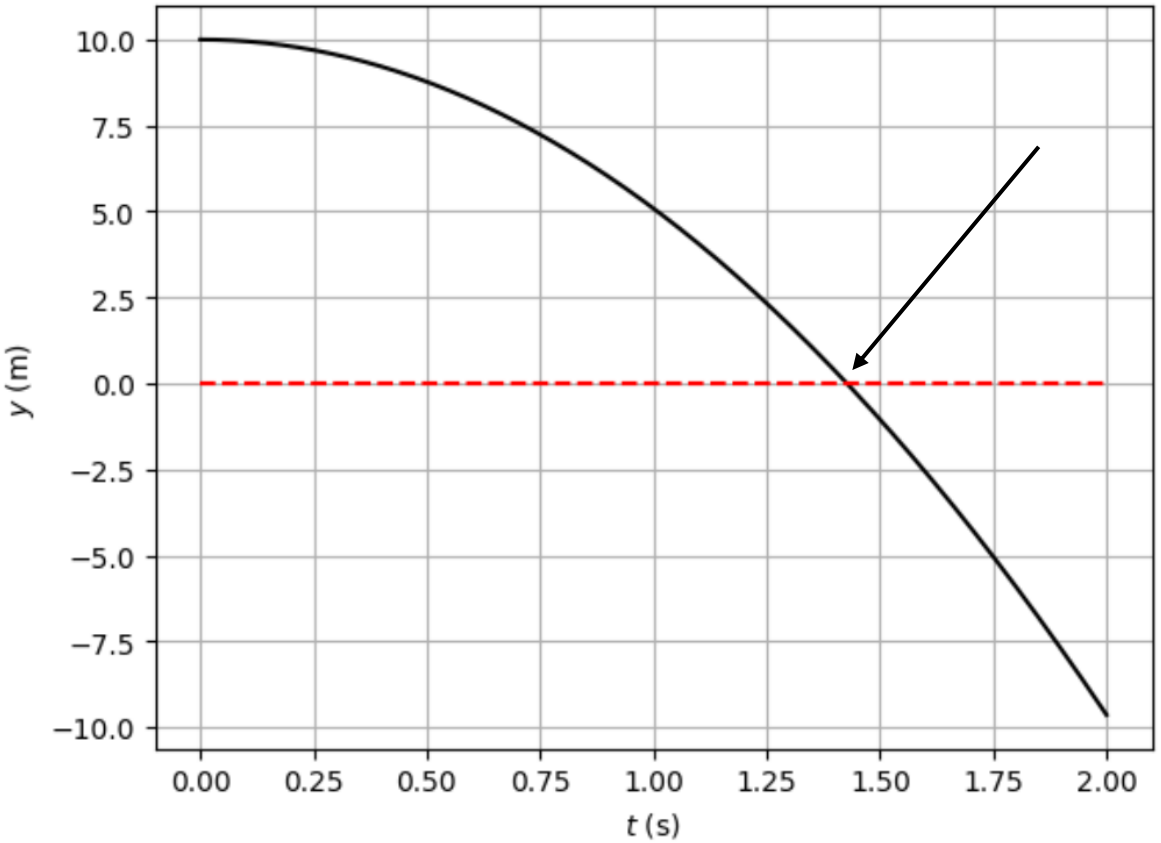
solve_ivp metode og events

Event når svingningen når maximum




```
def maximum(t,z):
    x,v = z
    return v
```

Event når banen af et projektil rammer jorden




```
def hit_ground(t,z):
    y,v = z
    return y
```

What is solve_ivp event flags/attributes? (select more)

Event when the oscillation reaches a maximum

```
def maximum(t,z):
    x,v = z
    return v
```

Event when the trajectory of a projectile hits the ground

```
def hit_ground(t,z):
    y,v = z
    return y
```

A. maximum.terminal=False 0%

B. hit_ground(terminal=False) 0%

C. maximum.terminal=None 0%

D. hit_ground.terminal=True 0%

E. maximum.direction=Max 0%

F. hit_ground.direction=0 0%

G. hit_gound.direction=-1 0%

What is solve_ivp event flags/attributes? (select more)

Event when the oscillation reaches a maximum

```
def maximum(t,z):
    x,v = z
    return v
```

Event when the trajectory of a projectile hits the ground

```
def hit_ground(t,z):
    y,v = z
    return y
```

A. maximum.terminal=False ##.##%

B. hit_ground(terminal=False) ##.##%

C. maximum.terminal=None ##.##%

D. hit_ground.terminal=True ##.##%

E. maximum.direction=Max ##.##%

F. hit_ground.direction=0 ##.##%

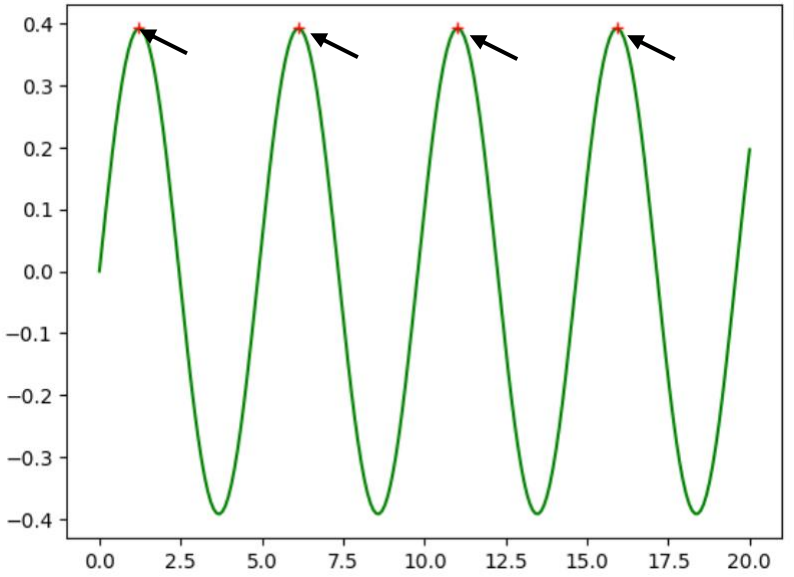
G. hit_gound.direction=-1 ##.##%

Svingninger

```
 1  import numpy as np
 2  from scipy.integrate import solve_ivp
 3  m = 0.25
 4  k1 = 0.4
 5  k3 = 0.1
 6  def f(t,z):
 7      x,v = z
 8      return [v,-k1*x/m-k3*x**3/m]
 9  def maximum(t,z):
10      x,v = z
11      return v
12  maximum.direction = -1
13  sol = solve_ivp(f,t_span=[0,20],y0=[0.0,0.5],events=[maximum],atol=1e-8,rtol=1e-8)
14  display(sol)
```

```
  message: The solver successfully reached the end of the integration interval.
  success: True
   status: 0
        t: [ 0.000e+00  1.215e-02 ...  1.992e+01  2.000e+01]
        y: [[ 0.000e+00  6.077e-03 ...  1.605e-01  1.959e-01]
            [ 5.000e-01  4.999e-01 ...  4.568e-01  4.339e-01]]
      sol: None
 t_events: [array([ 1.224e+00,  6.122e+00,  1.102e+01,  1.592e+01])]
 y_events: [array([[ 3.916e-01,  4.857e-17],
                   [ 3.916e-01,  2.845e-16],
                   [ 3.916e-01,  4.857e-17],
                   [ 3.916e-01, -5.447e-16]])]
     nfev: 1280
     njev: 0
      nlu: 0
```

```
15  import matplotlib.pyplot as plt
16  plt.figure()
17  plt.plot(sol.t,sol.y[0],'g-')
18  plt.plot(sol.t_events[0],sol.y_events[0][:,0],'r+')
```




```
1  print(sol.t_events[0])
```

```
[ 1.22437054  6.12185272 11.01933491 15.9168171 ]
```

```
1  print(sol.t_events[0][1:]-sol.t_events[0][0:-1])
```

```
[4.89748218 4.89748219 4.89748219]
```

Break

10 min

9 min

8 min

7 min

6 min

5 min

4 min

3 min

2 min

1 min

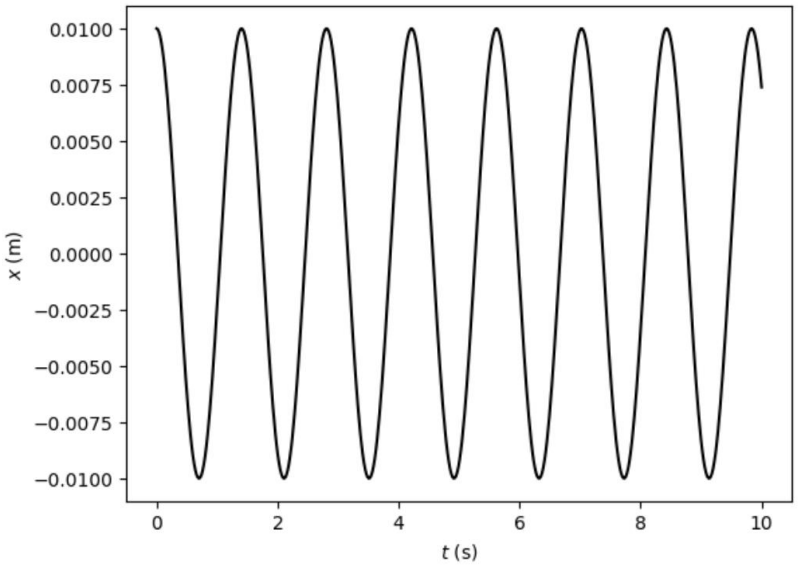
Time!

Svingninger

$$m\ddot{x} + b\dot{x} + kx = A\sin(\omega t)$$ (generelt)

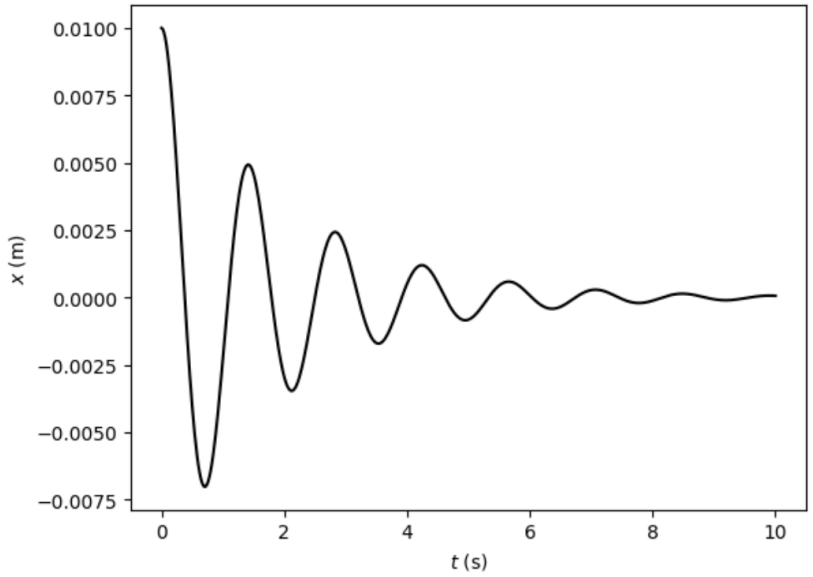
No damping and no forcing, $\omega = \sqrt{\frac{k}{m}}$

$$m\ddot{x} + b\dot{x} + kx = A\sin(\omega t)$$



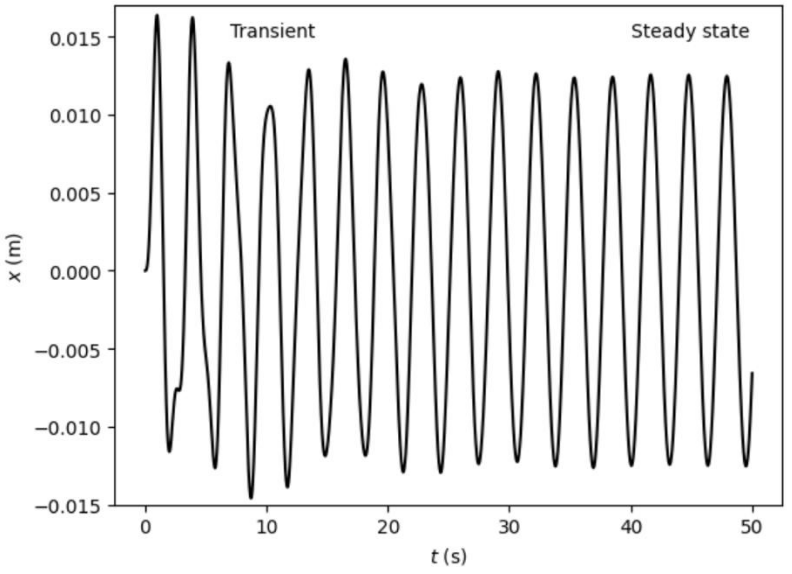
Damping and no forcing.

$$m\ddot{x} + b\dot{x} + kx = A\sin(\omega t)$$



Time plot of forced oscillation

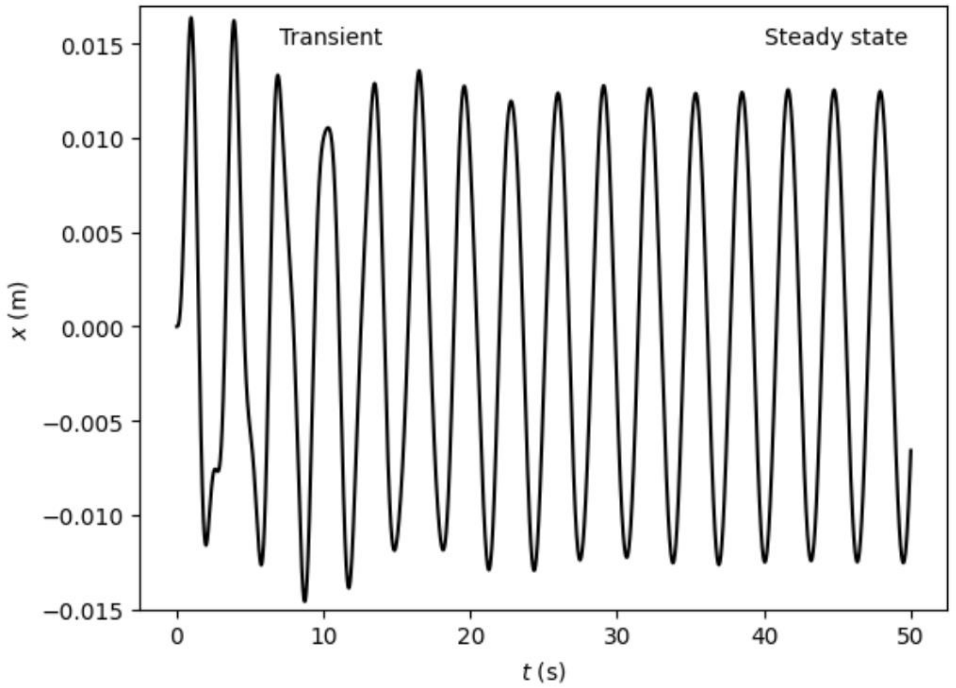
$$m\ddot{x} + b\dot{x} + kx = A\sin(\omega t)$$



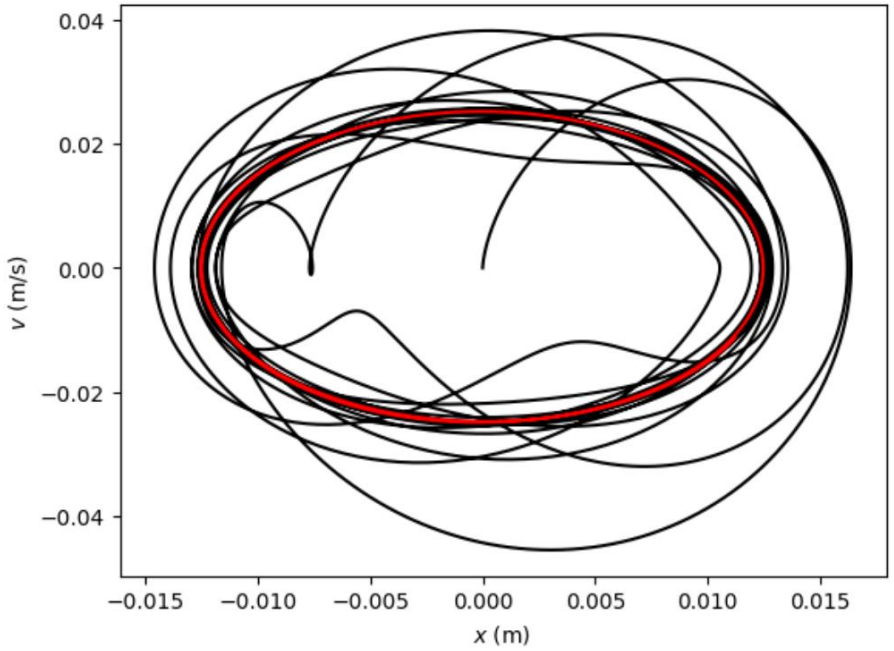
Forcerede svingninger

$$m\ddot{x} + b\dot{x} + kx = A\sin(\omega t)$$

Time plot of forced oscillation



Phase plane (x, v).



Resonans

$$m\ddot{x} + b\dot{x} + kx = A\sin(\omega t)$$

How does the amplitude depend on the frequency of forcing?

```python
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt
m = 0.50
k = 10.0
b = 0.1
A = 0.1
omega = 2.0
def f(t,z):
    x,v = z
    dxdt = v
    dvdt = (A*np.sin(omega*t)-k*x-b*v)/m
    return [dxdt,dvdt]
```

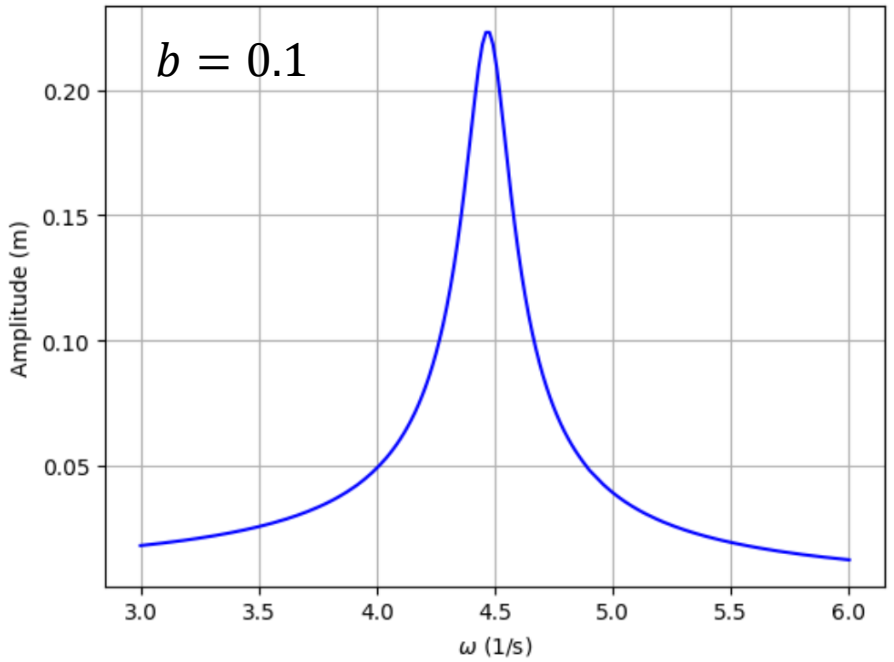
```python
transient = 100.0
duration = 20.0
t_eval = np.linspace(transient,transient+duration,500)
t_span = [0,transient+duration]
```

Skip transient

```python
omegas = []
amps = []
for omega in np.linspace(3.0,6.0,200):
    sol = solve_ivp(f,t_span=t_span,y0=[0,0],t_eval=t_eval,rtol=1e-8,atol=1e-8)
    t = sol.t
    x = sol.y[0]
    v = sol.y[1]
    omegas.append(omega)
    amps.append(np.max(x))
```

← Vary angular frequency

← Compute and add amplitude



Eksempel

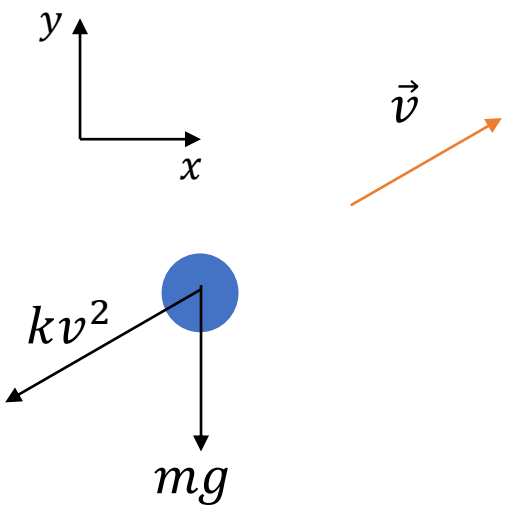
Projektilbevægelse med luftmodstand

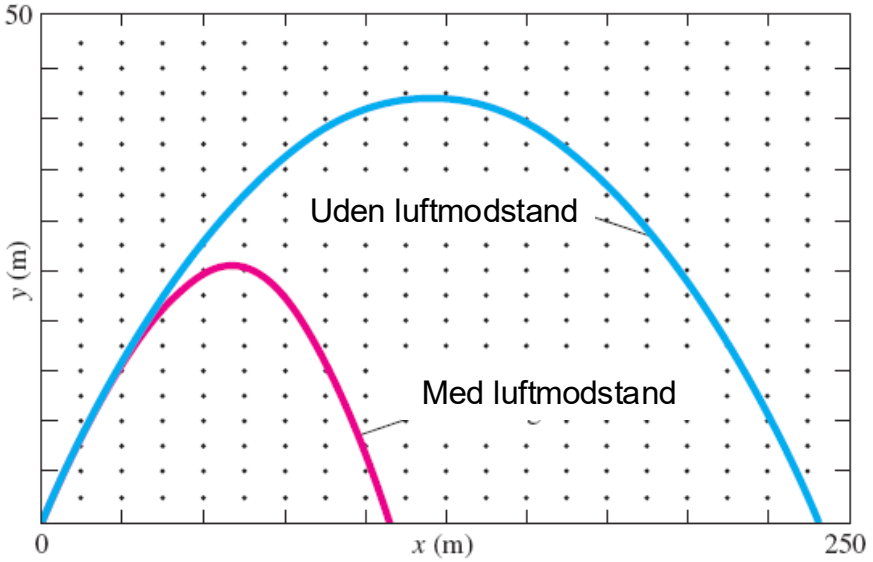
Luft- og vandmodstand (drag)

$$f = D \cdot v^2$$

$$D = \frac{1}{2} \cdot \rho_{\text{luft}} \cdot C \cdot A$$

Hvordan ser bevægelsesligningerne ud?





Skråt kast med luftmodstand

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
g = 9.82
m = 0.2
k = 0.002
v0 = 15.0
theta = 57.0*np.pi/180.0
def f(t,z):
    x,y,vx,vy = z
    v = np.sqrt(vx**2 + vy**2)
    dxdt = vx
    dydt = vy
    dvxdt = (-k*v*vx)/m
    dvydt = (-m*g-k*v*vy)/m
    return [dxdt,dydt,dvxdt,dvydt]
```

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
g = 9.82
m = 0.2
k = 0.002
v0 = 15.0
theta = 57.0*np.pi/180.0
def f(t,z):
    x,y,vx,vy = z
    v = np.sqrt(vx**2 + vy**2)
    dxdt = vx
    dydt = vy
    dvxdt = (-k*v*vx)/m
    dvydt = (-m*g-k*v*vy)/m
    return [dxdt,dydt,dvxdt,dvydt]
```

Projectile motion with air resistance. How is the apex (top point) found and when does it hit the ground?

A

```python
def top(t,z):
    x,y,vx,vy=z
    return vy
top.terminal=True
top.direction=-1
```

A. 0%

B

```python
def hit_ground(t,z):
    x,y,vx,vy=z
    return x
hit_ground.terminal=True
hit_ground.direction=-1
```

B. 0%

C

```python
def ground(t,z):
    x,y,vx,vy=z
    return y
ground.terminal=True
ground.direction=-1
```

C. 0%

D

```python
def hit_ground(t,z):
    x,y,vx,vy=z
    return y
hit_ground.terminal=True
hit_ground.direction=-1
```

D. 0%

E

```python
def top(t,z):
    x,y,vx,vy=z
    return y
top.terminal=True
top.direction=1
```

E. 0%

F

```python
def h(t,z):
    x,y,vx, vy=z
    return vy
h.terminal=False
h.direction=-1
```

F. 0%

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
g = 9.82
m = 0.2
k = 0.002
v0 = 15.0
theta = 57.0*np.pi/180.0
def f(t,z):
    x,y,vx,vy = z
    v = np.sqrt(vx**2 + vy**2)
    dxdt = vx
    dydt = vy
    dvxdt = (-k*v*vx)/m
    dvydt = (-m*g-k*v*vy)/m
    return [dxdt,dydt,dvxdt,dvydt]
```

Projectile motion with air resistance. How is the apex (top point) found and when does it hit the ground?

A
```python
def top(t,z):
    x,y,vx,vy=z
    return vy
top.terminal=True
top.direction=-1
```

B
```python
def hit_ground(t,z):
    x,y,vx,vy=z
    return x
hit_ground.terminal=True
hit_ground.direction=-1
```

C
```python
def ground(t,z):
    x,y,vx,vy=z
    return y
ground.terminal=True
ground.direction=-1
```

D
```python
def hit_ground(t,z):
    x,y,vx,vy=z
    return y
hit_ground.terminal=True
hit_ground.direction=-1
```

E
```python
def top(t,z):
    x,y,vx,vy=z
    return y
top.terminal=True
top.direction=1
```

F
```python
def h(t,z):
    x,y,vx, vy=z
    return vy
h.terminal=False
h.direction=-1
```

A. ##.##%

B. ##.##%

C. ##.##%

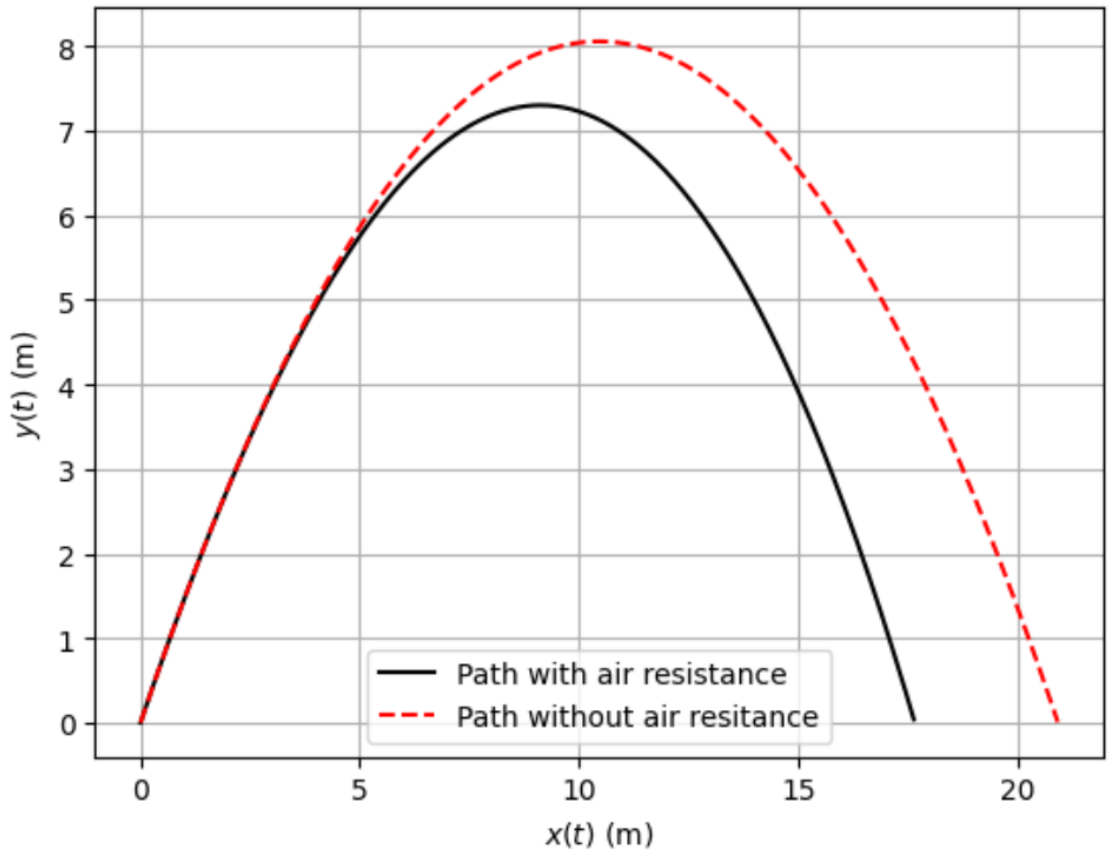
D. ##.##%

E. ##.##%

F. ##.##%

Skråt kast med luftmodstand

```
22  t = np.linspace(0,3,500)
23  sol = solve_ivp(f,t_span=[t[0],t[-1]],t_eval=t,y0=[0,0,v
24  x = sol.y[0]
25  y = sol.y[1]
26  plt.plot(x,y,'k-',label='Path with air resistance')
27  plt.xlabel('$x(t)$ (m)')
28  plt.ylabel('$y(t)$ (m)')
29  t = np.linspace(0,3,1000)
30  u = v0*np.cos(theta)*t
31  v = v0*np.sin(theta)*t-0.5*g*t**2
32  u1 = [X for X,Y in zip(u,v) if Y >= 0.0]
33  v1 = [Y for X,Y in zip(u,v) if Y >= 0.0]
34  plt.plot(u1,v1,'r--',label='Path without air resitance')
35  plt.legend()
36  plt.grid()
37  plt.show()
```



Select points on parabola above ground.

Quiz

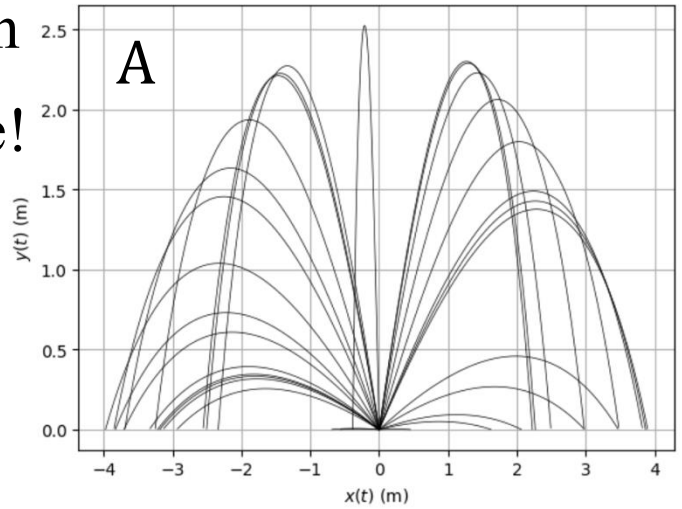
Projectile motion with air resistance. What varies?
Match letter with number

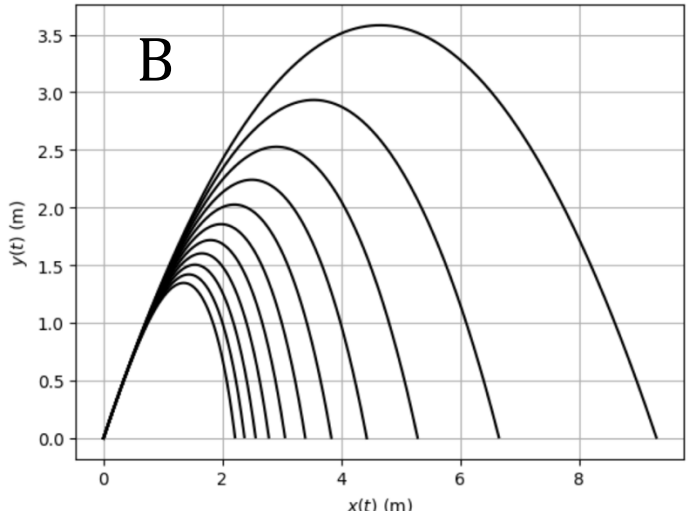
3 min

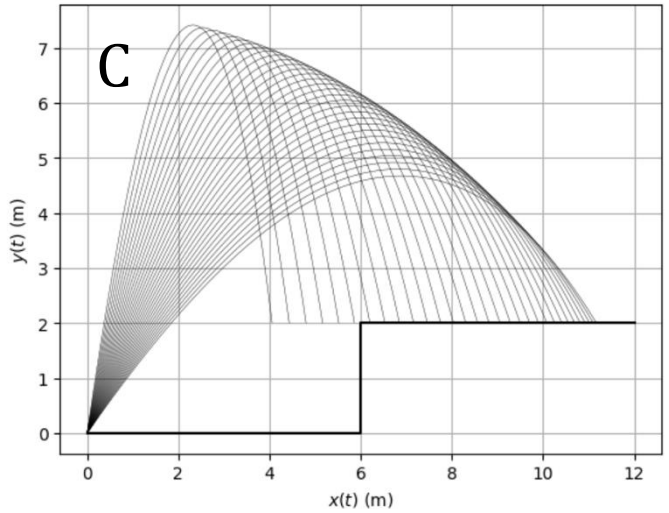
2 min

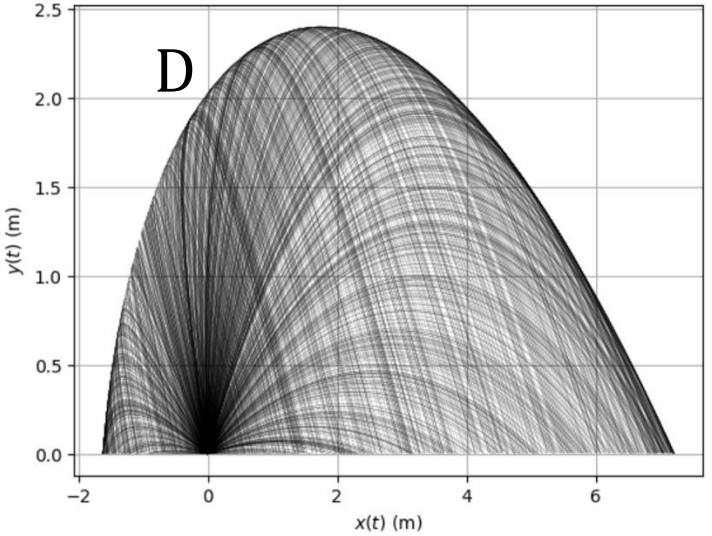
1 min

Time!









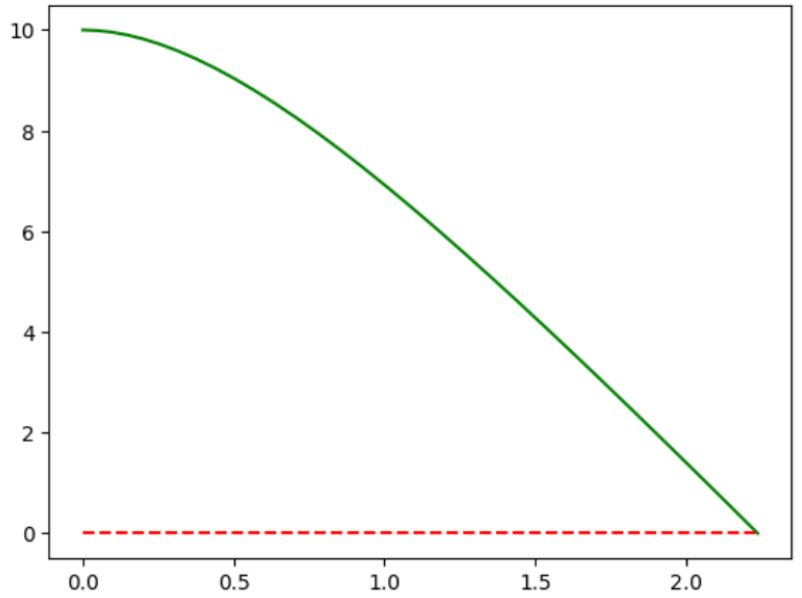
1. Gravity
2. Wind to the left
3. Airresistantce
4. Wind to the right
5. Initial speed
6. Angle

Fald med luftmodstand

The time the ground is hit:
t_events[0][0]

```
 message: A termination event occurred.
 success: True
  status: 1
       t: [ 0.000e+00  6.176e-03 ...  2.142e+00  2.237e+00]
       y: [[ 1.000e+01  1.000e+01 ...  5.633e-01 -8.882e-16]
           [ 0.000e+00 -6.035e-02 ... -5.938e+00 -5.966e+00]]
     sol: None
t_events: [array([ 2.237e+00])]
y_events: [array([[-8.882e-16, -5.966e+00]])]
    nfev: 182
    njev: 0
     nlu: 0
```

```python
import numpy as np
from scipy.integrate import solve_ivp
g = 9.82
m = 0.25
k = 0.4
h = 10.0
def f(t,z):
    y,v = z
    return [v,-g-k*v/m]
def hit_ground(t,z):
    y,v = z
    return y
hit_ground.direction = -1
hit_ground.terminal = True
sol = solve_ivp(f,t_span=[0,10],y0=[h,0],events=[hit_ground],atol=1e-8,rtol=1e-8)
display(sol)
print(sol.t_events[0][0])
```

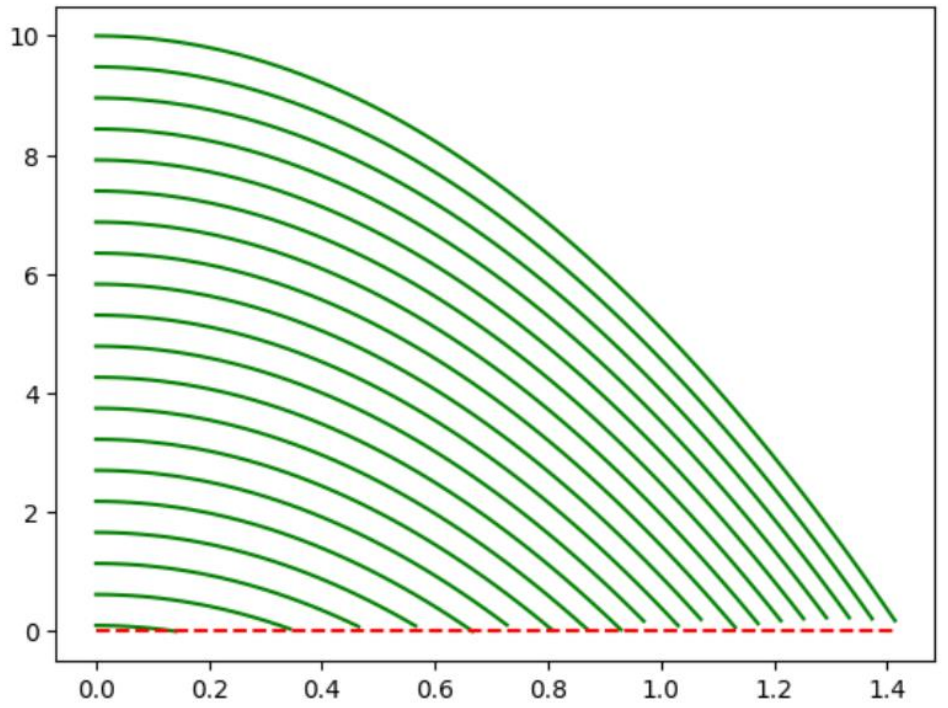


Fald med luftmodstand

No air resistance yet

For different heights

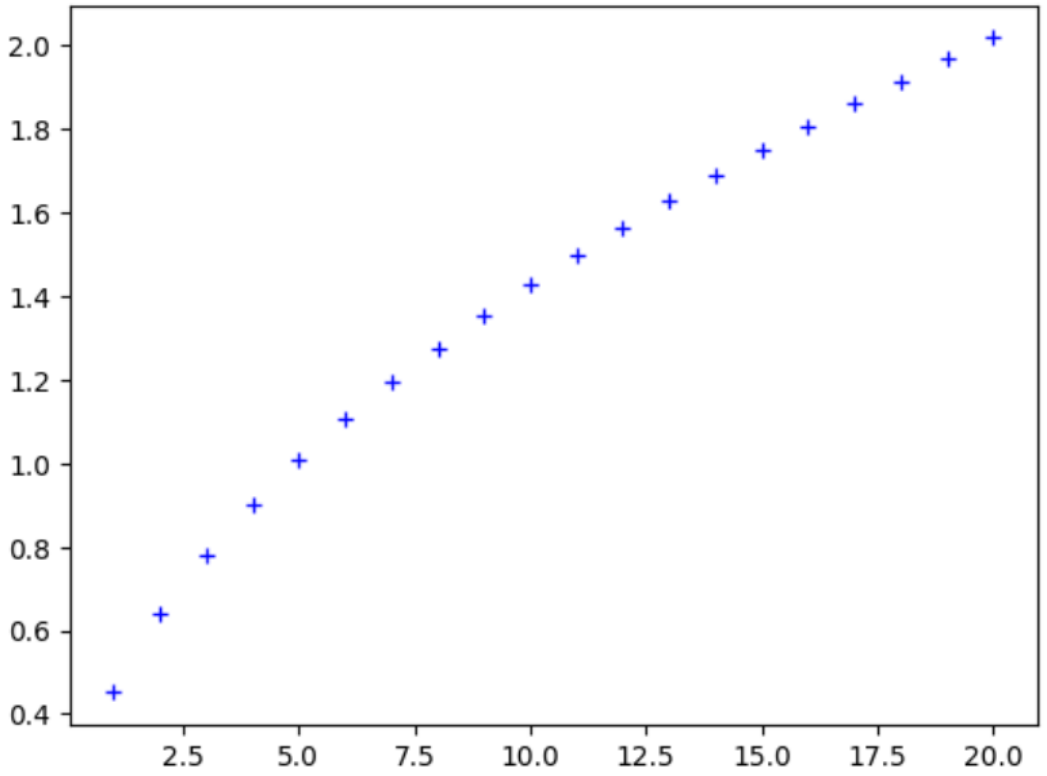
```
1  k = 0
2  t_eval = np.linspace(0,2,100)
3  plt.figure()
4  for h in np.linspace(0.1,10,20):
5      sol = solve_ivp(f,t_span=[0,10],y0=[h,0],t_eval=t_eval,events=[hit_ground],atol=1e-8,rtol=1e-8)
6      plt.plot(sol.t,sol.y[0],'g-')
7  plt.plot([sol.t[0],sol.t[-1]],[0,0],'r--')
8  plt.show()
```



Fald med luftmodstand

```
1  h_values = []
2  t_values = []
3  for h in np.linspace(1,20,20):
4      sol = solve_ivp(f,t_span=[0,10],y0=[h,0],events=[hit_ground],atol=1e-8,rtol=1e-8)
5      h_values.append(h)
6      t_values.append(sol.t_events[0][0])
7  plt.figure()
8  plt.plot(h_values,t_values,'b+')
9  plt.show()
```

Save the height and time



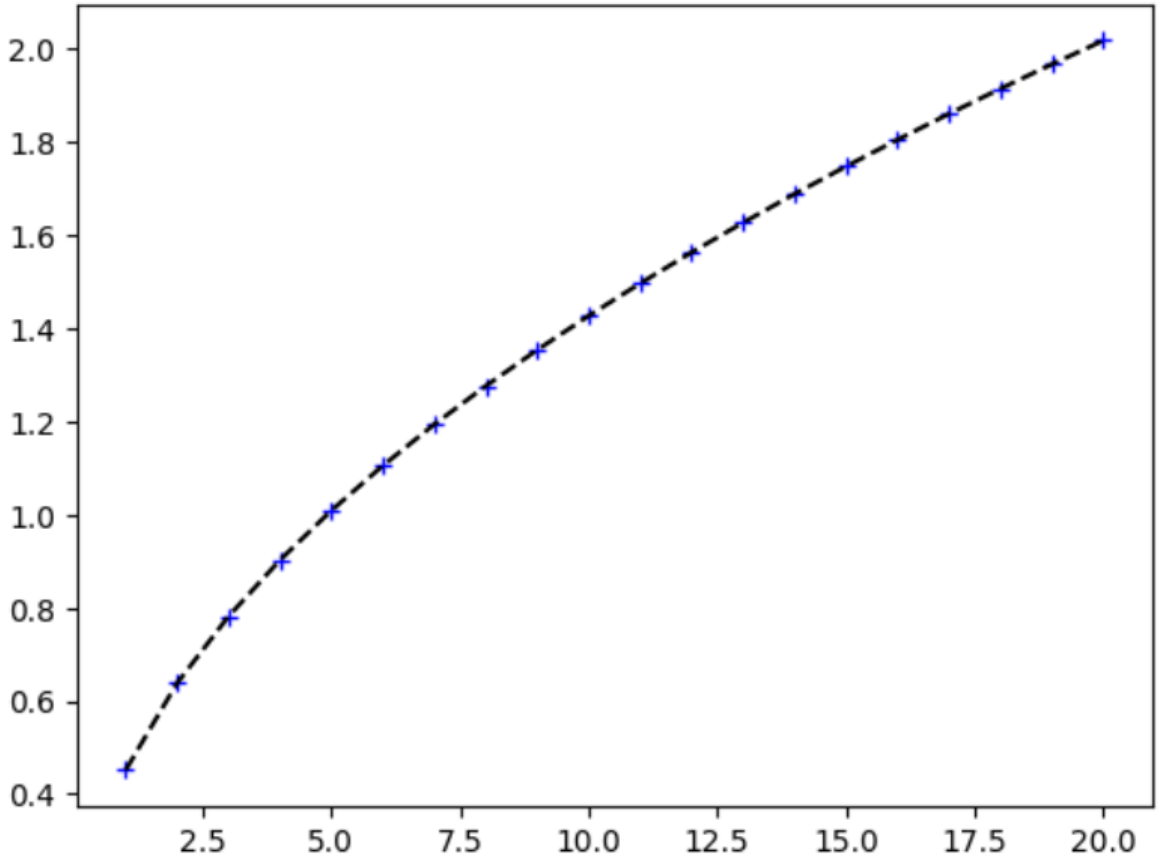
Fald med luftmodstand

```
1  from scipy.optimize import curve_fit
2  def fit(t,A,B):
3      return A*t**B
4  param,cov = curve_fit(fit,h_values,t_values)
5  display(param)
6  display(np.sqrt(np.diag(cov)))
```

```
array([0.45129368, 0.5       ])

array([7.83131354e-17, 6.75349626e-17])
```

Looks good, small uncertainties



Fald med luftmodstand

Now, with air resistance

For different heights

```
1  k = 0.4
2  h_values = []
3  t_values = []
4  for h in np.linspace(1,20,20):
5      sol = solve_ivp(f,t_span=[0,10],y0=[h,0],events=[hit_ground],atol=1e-8,rtol=1e-8)
6      h_values.append(h)
7      t_values.append(sol.t_events[0][0])
8  plt.figure()
9  plt.plot(h_values,t_values,'b+')
10 plt.show()
```

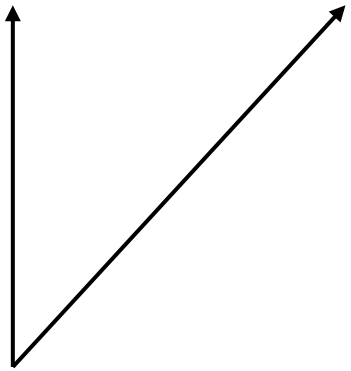
Save the height and time

Fald med luftmodstand

```
1  from scipy.optimize import curve_fit
2  def fit(t,A,B):
3      return A*t**B
4  param,cov = curve_fit(fit,h_values,t_values)
5  display(param)
6  display(np.sqrt(np.diag(cov)))
```

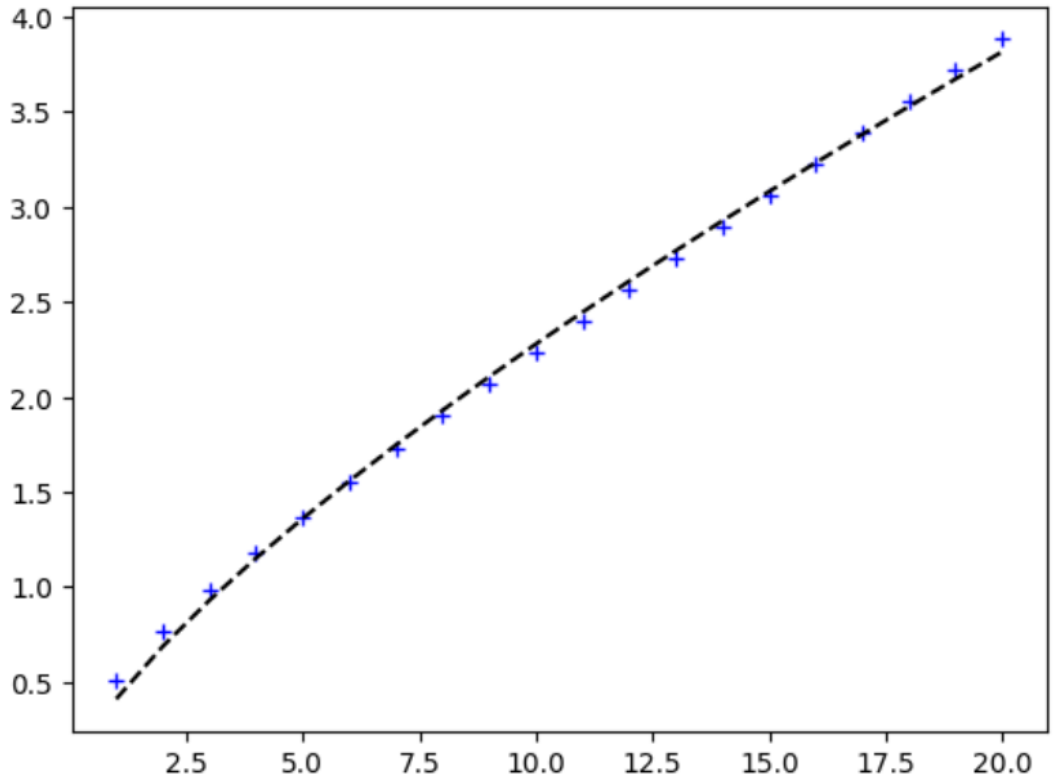
```
array([0.41078282, 0.74409693])

array([0.01123085, 0.0103219 ])
```



Looks ok, smallish uncertainties

There seems to be some systematic deviations



Kursusoverblik

Overblik over forårets forelæsninger, **foreløbig** plan:

Uge 1: Kinematik 1D (Kap. 3)
Uge 2: Kinematik 2D (Kap. 4)
Uge 3: Usikkerhed
Uge 4: Eksperimenter
Uge 5: Kræfter 1 (Kap. 5)
Uge 6: Kræfter 2 (Kap. 6)
Uge 7: Bevægelsesligninger (Kap. 15)
Uge 8: Arbejde Og Energi (Kap. 7)
Påske (30/3)
Påske (6/4)
Uge 9: Energibevarelse (Kap. 8)
Uge 10: Stød Og Impulsbevarelse (Kap. 9)
Uge 11: Rotation 1 (Kap. 10)
Uge 12: Rotation 2 (Kap. 11)
Uge 13: Energitema 1

Lærebogen er University Physics fra OpenStax, der er gratis tilgængelig på https://openstax.org/details/books/university-physics-volume-1 og https://openstax.org/details/books/university-physics-volume-2.

Link til boghandel:
https://polyteknisk.dk/home/dtu/soege_resultat?utf8=%E2%9C%93&q=10060&b=20&st=c&code=&exact_match=false

Gode studie vaner:
- Læs kapitel inden forelæsning,
- Forelæsning
- Gruppe regning (lær af dine medstuderende)
- Færdiggør gruppe øvelser hjemme.

Repetition (3 gange) & øvelse er nøglen!